and it can do that too. But now ask, "What did our company decide about laptop reimbursements last month?" Unless that information is supplied to it, the model cannot know.

Ask it to check current inventory, create a calendar event or calculate a figure from a live spreadsheet. The model does not magically possess access to those systems.

This distinction is fundamental: a language model generates language; an AI system surrounds that model with context and capabilities.

A simple local chat setup might look like this:

User → prompt → model → response

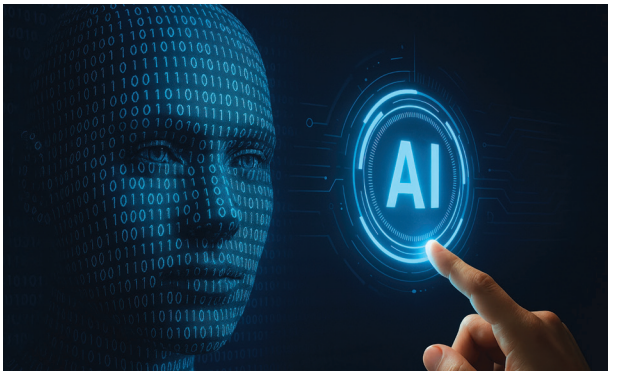
A useful AI application usually looks closer to this:

User → interface → application logic → instructions → model → knowledge/tools → validation → response

The interface makes the system usable; application logic decides what happens; instructions control behaviour; retrieval provides outside information; tools enable actions; validation checks results; and memory carries useful state between turns.

Even "chat" is an application convention. Underneath, chat-tuned causal language models still process sequences of tokens. Hugging Face's Transformers documentation explains that chat templates convert role-based messages such as system, user and assistant turns into the token sequence the particular model expects. Different model families can require different control tokens and formatting, so copying a prompt wrapper from one model to another can produce weaker behaviour.

The first lesson of building your own AI, therefore, is to start thinking in systems.



The seven pieces around your model

You do not need every component in every project, but it helps to know what each one does.

Instructions define the job. A system message might tell the model that it is a support assistant, that answers must be concise, or that it must never invent a source. Instructions shape behaviour within the supplied context; they do not permanently change the model.

Context is everything the model can see for the current request: conversation history, instructions, documents and tool results. Context is temporary. Remove it and the model does not automatically retain it.

Knowledge retrieval fetches relevant information from outside the model. Instead of stuffing an entire handbook into every prompt, an application can search it and send only relevant passages. This is the core idea behind retrieval-augmented generation, or RAG.

Memory stores useful state beyond the immediate prompt: perhaps a preference, previous project decision or summary of an older conversation. Your application decides what to save, where to save it and when to retrieve it.

Tools allow the system to do things the model cannot reliably do through text generation alone. A calculator performs arithmetic; a function can query a database or rename a file. Hugging Face's current tool-use documentation makes an important architectural point: a tool-capable model does not itself execute a function. It requests a tool call; your application executes it and returns the result.

Structured output turns prose into data software can consume. Instead of saying "The invoice total is ₹18,450," an application may require JSON fields for invoice number, subtotal, tax and total.

Finally, **evaluation and guardrails** determine whether the system did what you intended. Did retrieval find the right document? Did the model cite it? Did it call the correct tool? Did the result obey the schema?

Put together, these layers turn a generic model into a specialised product.

Four ways to make any AI model yours

Most custom AI projects rely on four broad techniques: prompting, retrieval, fine-tuning and tools. They complement one another.

Prompting: change the instructions

Prompting is the cheapest and fastest method. You keep the model unchanged and give it clearer instructions, examples or constraints.

Imagine you want an AI to turn meeting notes into an internal update containing "Decision", "Owner", "Deadline" and "Risk". Start with a system instruction explaining the format. Add one or two examples if necessary. If the model reliably follows the pattern, you may not need anything more elaborate.

Prompting works best when the model already has the capability and mainly needs steering. It is less suitable when the missing ingredient is private or rapidly changing knowledge.

RAG: change what the model can see

Suppose the AI must answer questions about your company's 400-page product catalogue. Repeatedly pasting the catalogue into a prompt is inefficient, while fine-tuning on it creates another problem: specifications change.